What makes a good PR

Use this when evaluating source PRs for Helix task quality. The ideal task has meaningful complexity, clean verifiability through tests, and enough context that a fellow can understand the problem without spending days just figuring out what's going on.

What to look for

Lines of code changed[1] — Target 25–500 SLOC. Under 25 is usually trivial. Over 1000 is too large for a fellow to digest and work with effectively. Check the actual diff; lock file changes and boilerplate don't count.

1. Check the actual diff — generated code doesn't count

Existing, separated tests[2] — The PR should include new tests in a dedicated test file. Not test modifications only, but new tests that exist because of this feature or fix.

2. You'll need these for F2P tests

Strong PR body[3] — The original author explains the problem, the approach, and the testing strategy. A well-documented PR makes it much easier to write a high-quality LLM prompt later.

3. This becomes the foundation for the problem_statement

Watch out for these

Good

PR adds a new caching layer with 3 new test files covering cache hits, misses, and eviction. Diff is 180 SLOC of real logic. PR body explains the performance problem and the approach.

Bad

PR upgrades lodash from 4.17.20 to 4.17.21. Diff is 2000 lines but it's all lock file changes. No new tests because the goal is just to keep everything working.

> High SLOC from generated code or dependency upgrades doesn't mean meaningful complexity. Look at the actual logic changes.

Awkward test changes — If the test changes are just updating function call signatures to match a refactor, they aren't real F2P tests. There's no meaningful new behavior being

verified.

Previous

Next